Project note: How my chatbot is designed

Topic: AI-powered industrial document chatbot
Name: Deep Bisen

1. The user enters a question in the Python CLI.
2. The program reads the sample technical documents.
3. It splits the text into small, labelled chunks.
4. TF-IDF finds chunks that share important query terms.
5. The best excerpts are placed in a grounded prompt.
6. FLAN-T5-small is the selected Hugging Face model.
7. The model is asked to answer only from the excerpts.
8. Retrieved filenames help the reader check the evidence.

Why retrieval? A model may not know our local manual.
Adding the relevant text gives it useful context.
Why prompt engineering? It avoids an expensive training run.
Fine-tuning is optional and was not performed here.

Important: sample maintenance values are educational.
Actual machine work must follow approved site procedures.

Project note: Testing and what remains

The retrieval tests were actually run in Python.
Five sample questions retrieved the correct topic first.
An unrelated geography question returned no evidence.
The offline CLI printed source text with a clear label.

This offline mode is extractive. It does not run an LLM.
Hugging Face access returned a proxy 403 error here.
Therefore no generated-answer accuracy is claimed.

To complete the practical demonstration:
1. Install torch, transformers and the listed dependencies.
2. Download google/flan-t5-small on an allowed network.
3. Run the CLI without the --extractive option.
4. Ask bearing, quality and missing-information questions.
5. Check every answer against the supplied documents.
6. Capture real model interaction screenshots.

Final check: fill roll number, college and semester.
Upload the report, source, sample files and real evidence.
Keep unsupported facts out of the final results.